

Gestión de la Información en la Web
Curso 2026-27
Práctica 3 – Formato XML

Fecha de entrega: domingo 18 de octubre de 2026

Entrega de la práctica

La entrega de la práctica se realizará a través del Campus Virtual de la asignatura mediante un fichero `pr3.py`. El esqueleto de este fichero se puede descargar del Campus Virtual.

Lenguaje de programación

Python 3.13 o superior.

Calificación

Se medirá la corrección mediante tests de unidad. Además de la corrección, se valorará la **calidad, concisión y claridad del código**, la incorporación de **comentarios** explicativos, su **eficiencia** tanto en tiempo como en memoria y la puntuación obtenida en Pylint.

Declaración de autoría e integridad

Todos los ficheros entregados contendrán una cabecera en la que se indique la asignatura, la práctica, el grupo y los autores. Esta cabecera también contendrá la siguiente declaración de integridad:

Declaramos que esta solución es fruto exclusivamente de nuestro trabajo personal. No hemos sido ayudados por ninguna otra persona o sistema automático ni hemos obtenido la solución de fuentes externas, y tampoco hemos compartido nuestra solución con otras personas de manera directa o indirecta. Declaramos además que no hemos realizado de manera deshonesto ninguna otra actividad que pueda mejorar nuestros resultados ni perjudicar los resultados de los demás.

No se corregirá ningún fichero que no venga acompañado de dicha cabecera.

El objetivo de esta práctica es crear una serie de funciones para extraer información a partir de un fichero XML con información sobre los restaurantes de la ciudad de Madrid. Para ello se va a usar el catálogo «Restaurantes con perfil turístico de la ciudad de Madrid» del portal de datos abiertos del Ayuntamiento de Madrid. El conjunto de datos incluye información sobre restaurantes de la ciudad de Madrid divididos por tipos de cocina, especialidades, servicios, etc. Así mismo, se incluyen los datos básicos de cada restaurante, una descripción, su geoposición y dirección postal, así como un conjunto de fotografías relativas a cada punto. En los casos en los que aplica, se ofrecen horarios de apertura y costes de acceso si los tuviera. La información se encuentra en: <https://datos.madrid.es/dataset/300033-0-turismo-restauracion/information>, y el conjunto de datos XML que hay que procesar está situado en: <https://datos.madrid.es/dataset/300033-0-turismo-restauracion/downloads>. Junto al enunciado de la práctica se proporciona el fichero XML a tratar y una versión más fácil de leer *por humanos* `restaurantes_v1_es_pretty.xml`.

Antes de empezar a escribir las funciones, el primer paso será revisar el fichero XML y entender la estructura de sus entradas.

Funciones para extraer información del fichero XML

1) Nombres de restaurantes [2pt]

Escribe una función `nombres_restaurantes(filename)` **usando SAX** que recibe la ruta del fichero XML y devuelve una **lista ordenada** alfabéticamente con los nombres de todos los restaurantes que aparecen. Ten en cuenta que el fichero XML contiene texto HTML escapado (por ejemplo tildes y otros símbolos) que deberás «desescapar». Para ello utiliza la función `html.unescape` en la biblioteca predeterminada de Python `html`. **También debes eliminar los posibles espacios al principio y final que pueda haber en los nombres de restaurantes.**

Un ejemplo de la salida de esta función sería:

```
1 ['"Tres Peces" El Ventorrillo Murciano',
2  '100 Montaditos',
3  '11 Nudos Terraza Nordés',
4  '19 Sushi Bar',
5  '80º',
6  '99 Sushi Bar (Hermosilla)',
7  ...
8 ]
```

2) Subcategorías de cocina [3pt]

Escribe una función `subcategorias(filename)` **usando SAX** que recibe la ruta del fichero XML y devuelve un **conjunto** de todas las subcategorías que existen en el fichero de restaurantes. Como las subcategorías pertenecen a una categoría, queremos generar cadenas del estilo `'CATEGORIA > SUBCATEGORIA'`.

Ten en cuenta que un único restaurante puede tener varias subcategorías, por ejemplo:

```
1 <categorias>
2   <categoria>
3     <item name="idCategoria">7102</item>
4     <item name="Categoria">Internacional</item>
5     <subcategorias>
6       <subcategoria>
7         <item name="idSubCategoria">7462</item>
8         <item name="SubCategoria">Fusión</item>
9       </subcategoria>
10      <subcategoria>
```

```
11         <item name="idSubCategoria">7146</item>
12         <item name="SubCategoria">Internacional</item>
13     </subcategoria>
14     <subcategoria>
15         <item name="idSubCategoria">7116</item>
16         <item name="SubCategoria">Japonesa</item>
17     </subcategoria>
18     <subcategoria>
19         <item name="idSubCategoria">7119</item>
20         <item name="SubCategoria">Peruana</item>
21     </subcategoria>
22 </subcategorias>
23 </categoria>
24 </categorias>
```

Este restaurante debería generar las siguientes 4 subcategorías en el conjunto:

- 'Internacional > Fusión'
- 'Internacional > Internacional'
- 'Internacional > Japonesa'
- 'Internacional > Peruana'

Un ejemplo de la salida de esta función sería:

```
1 {'Española > Andaluza',
2  'Española > Arrocería',
3  'Española > Asador / Parrilla',
4  'Española > Asturiana',
5  'Española > Balear',
6  'Española > Canaria',
7  ...
8 }
```

3) Información de un restaurante [2.5pt]

Escribe una función `info_restaurante(filename, name)` **utilizando DOM o ElementTree** que recibe la ruta del fichero XML y un nombre de restaurante y devuelve un diccionario Python con la información básica del restaurante:

- Nombre del restuarante
- Descripción (campo body)
- Correo electrónico
- Web
- Número de teléfono
- Horario

Si alguno de estos campos no existe, la función no debe fallar pero el diccionario contendrá `None` en esa entrada. De la misma manera, si no hay ningún restaurante con ese nombre se devolverá `None` en lugar de un diccionario. **Al igual que en apartados anteriores, debéis «desescapar» las**

cadenas que tengan texto HTML escapado y eliminar los espacios en blanco al principio y final de las cadenas leídas del fichero, si hubiera.

Un ejemplo de invocación a la función sería el siguiente diccionario:

```
1 { 'nombre': 'La Charca Restaurante',
2   'descripcion': '<p style="text-align:start"><strong>Situado junto a ...',
3   'email': None,
4   'web': 'https://www.esmadrid.com/restaurantes/charca-restaurante',
5   'phone': '(+34) 913 578 504',
6   'horario': '<p>Lun - Dom: 12:00 - 24:00 h</p>'
7 }
```

4) Restaurantes por cercanía [2.5pt]

Escribe una función `busqueda_cercania(filename, lugar, n)`, **utilizando DOM o Element-Tree**, que recibe la ruta del fichero XML, un lugar expresado como una cadena de texto soportada por GeoPY y un número n de kilómetros, y devuelve una lista de parejas (`distancia`, `nombre_restaurante`) con aquellos restaurantes que están como mucho a n kilómetros de distancia del lugar indicado. **La lista generada debe estar ordenada de lugar más cercano a más lejano**, y debes «desescapar» el texto HTML escapado en los nombres de los restaurantes. Para el manejo de lugares y distancias debes utilizar la biblioteca GeoPy, tal y como vimos en la práctica 2.

Un ejemplo del resultado invocación

```
1 busqueda_cercania('restaurantes_v1_es.xml',
2                   'Profesor José García Santesmases 9, Madrid, España', 2.5)
```

sería la siguiente lista:

```
1 [(1.5539711282210105, 'Restaurante Café de Oriente Museo del Traje'),
2  (1.656597184132381, 'Sal Gorda'),
3  (2.0115345641314426, 'Marmalé'),
4  (2.0150313290486888, 'Gobolem (San Francisco)'),
5  (2.054870466815955, 'José Luis (San Francisco)'),
6  (2.083714331520481, 'Desde 1911'),
7  (2.217346810397983, 'EMi Restaurante'),
8  (2.226196307883302, 'Naveira do mar'),
9  (2.324274686320881, 'Taberna del Chato'),
10 (2.32791467728629, 'Julita Café'),
11 (2.3921188926210353, 'Casa Adriano')]
```

Al igual que en apartados anteriores, debéis «desescapar» las cadenas que tengan texto HTML escapado y eliminar los espacios en blanco al principio y final de las cadenas leídas del fichero, si hubiera.